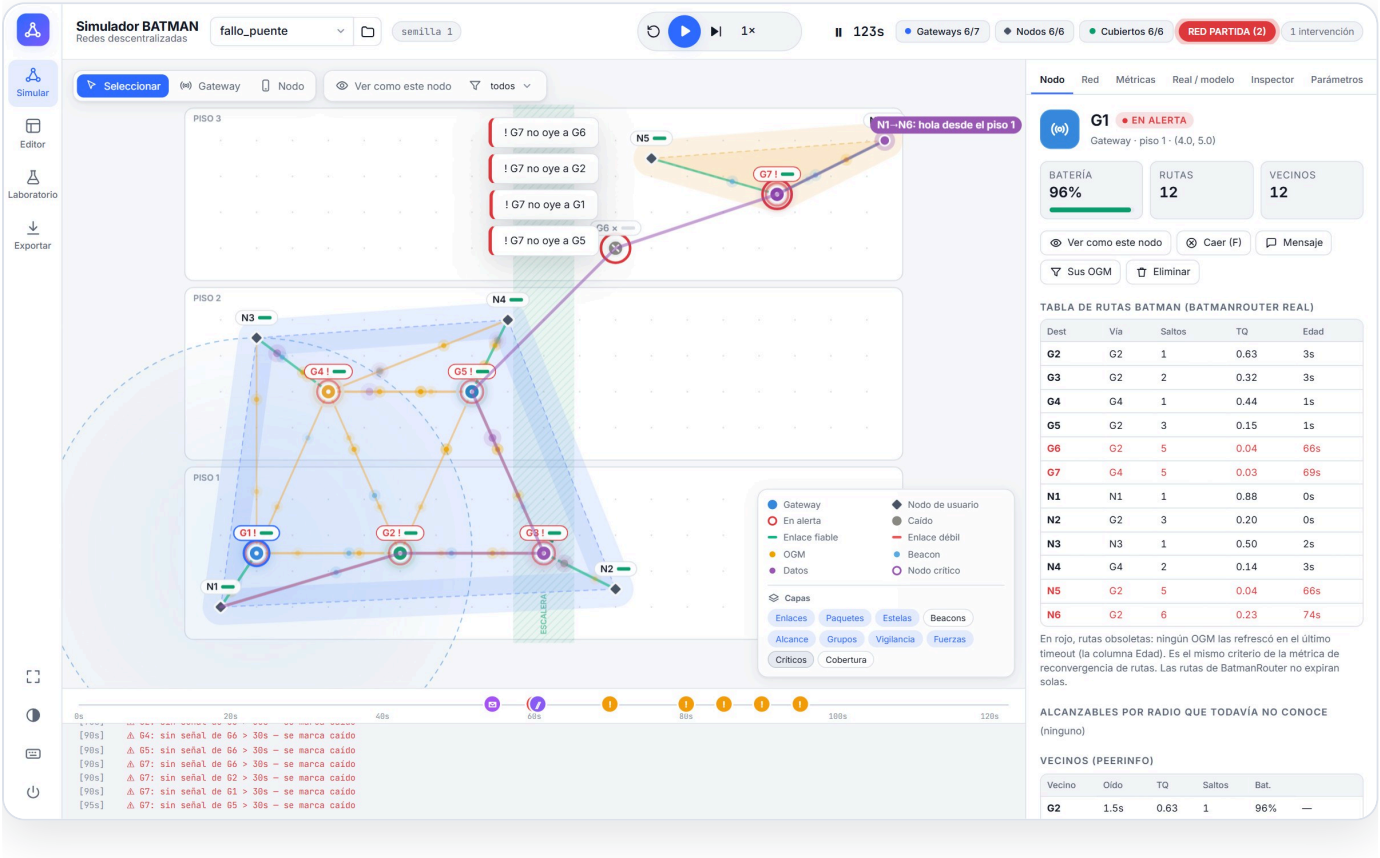
PROYECTO DE GRADO · GUÍA VISUAL

Simulador BATMAN, explicado fácil

Qué hace el proyecto, qué es cada cosa de la interfaz web y cómo leer cada métrica. Con capturas reales del simulador corriendo.



1 La idea en un minuto

2 Conceptos básicos (OGM, beacon, TQ...)

3 Cómo está armado el proyecto

4 La pantalla principal

5 El mapa y sus capas

6 Herramientas del mapa

7 El panel lateral, pestaña por pestaña

8 Historia de un fallo, paso a paso

9 Las 8 gráficas en vivo

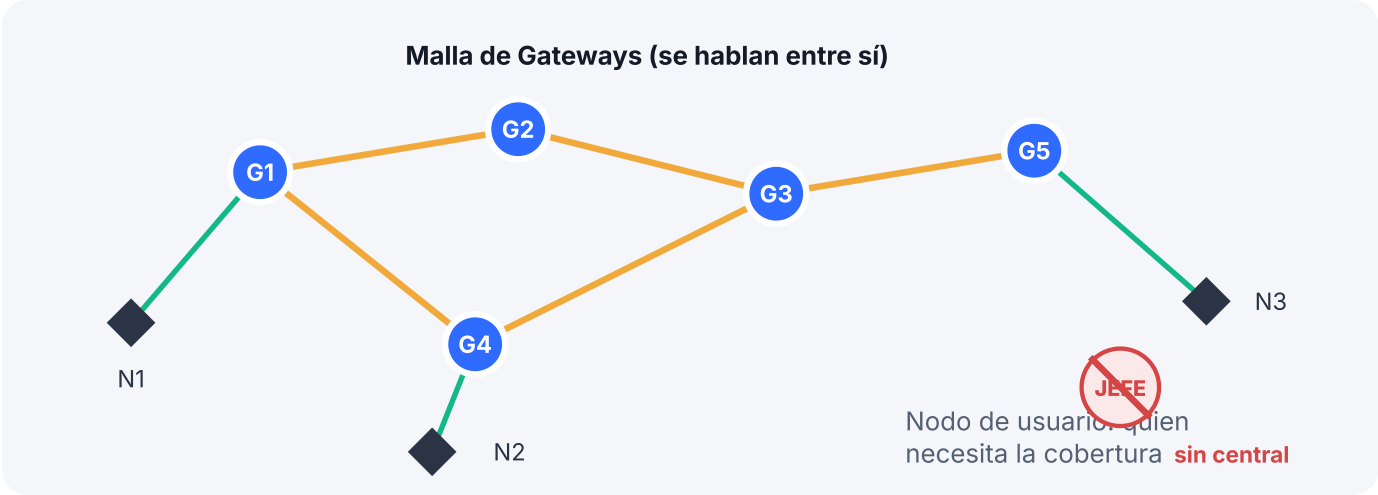
10 Las métricas del resumen, una por una

11 Laboratorio, editor y extras · Chuleta final

1 · LA IDEA EN UN MINUTO

¿Qué hace este proyecto?

Simula una red de radios que se organizan solos, sin una central que mande, y mide cuánto aguanta cuando algunos fallan o se mueven.



Gateway (G)
Un radio que forma la malla. Habla con otros Gateways y le da cobertura a los usuarios que tiene cerca. En el mapa es un **círculo**.

◆ Nodo de usuario (N)
Un aparato que necesita conectarse (un teléfono, un sensor). Se conecta a los Gateways que oye. En el mapa es un **rombo**.

La analogía que sirve para explicarlo

Imagina un grupo de personas con walkie-talkies en un edificio. Nadie tiene el mapa completo: cada uno sólo oye a los que tiene cerca y le pasa los recados a otro. Si una persona se va, los demás tienen que darse cuenta solos y buscar otro camino. El simulador mide **qué tan rápido se dan cuenta**, **cuánta gente se queda sin señal** y **cuántos mensajes llegan**.

Lo que lo hace especial

- **Corre el código real del protocolo.** Las rutas no las calcula una "maqueta": las calcula la misma clase `BatmanRouter` que correría en un radio de verdad (`mesh/router.py`). Lo único simulado es el mundo físico: posiciones, alcance de la señal y paquetes que se pierden.
- **Es descentralizado de verdad.** Cada nodo tiene su propia tabla y su propia idea de la red. La interfaz deja "ver la red como la ve un nodo".
- **Es reproducible.** Con la misma *semilla* la corrida sale idéntica, en la ventana, en la web o en la terminal.
- **Mide lo que importa para la tesis:** cobertura, particiones, detección de fallos, entrega de datos y qué tan correcto es lo que cada nodo "cree".

La pregunta que contesta

"¿Qué pasa con la cobertura y la conexión de la red si un radio falla (o se mueve)?"

2 · CONCEPTOS BÁSICOS

Las cinco palabras que hay que entender

Todo lo demás (la interfaz y las métricas) se apoya en estos conceptos.

OGM y beacon: los dos mensajes que mantienen viva la red

OGM · "aquí estoy" (se reenvía salto a salto)



Cada vecino lo reenvía una vez, restando 1 al TTL. Así todos aprenden por dónde se llega a A.

Beacon · "hola, soy yo" (un solo salto)

Lo oyen sólo los vecinos dentro del alcance; no se reenvía. Lleva batería y posición.



	OGM	Beacon
Para qué	Armar las rutas y medir su calidad (TQ)	Saber quién está al lado, su batería y su posición
Cada cuánto	4 s (`batman_cada`)	2 s (`beacon_cada`)
Hasta dónde llega	Toda la malla, máximo 6 saltos (TTL)	Sólo vecinos directos

TTL: el "contador de saltos"

Es cuántos saltos le quedan a un mensaje antes de descartarse. Evita que un mensaje dé vueltas para siempre. Un OGM sale con 6: un nodo a 7 saltos nunca se entera de ese origen. Los paquetes de datos tienen su propio TTL de 50.

Ruta

Para cada destino, cada nodo guarda **por qué vecino** mandarle las cosas (el "siguiente salto") y a cuántos saltos está. Esa es su **tabla de rutas**. No hay una tabla global: cada nodo tiene la suya.

TQ: la nota de calidad de una ruta

Últimos 16 OGM esperados de un vecino:



Llegaron 12 de 16 → **TQ del enlace = 0.75**

Ruta de 2 saltos: 0.9 × 0.75 = **0.68**

Entre 0 (pésima) y 1 (perfecta). Gana la ruta de mayor TQ.

La señal de radio se debilita con la distancia y los pisos

Cada vez que un nodo transmite, a cada vecino le llega el mensaje con una probabilidad (la **fiabilidad** del enlace):

```
fiabilidad = 1 - falloff × (distancia / alcance) - pérdida_base   (× atenuación por cada piso de diferencia)
```

Si la distancia supera el alcance (`rango_comm`), vale 0. Por eso un vecino "en rango" también pierde paquetes: a 2 m casi siempre recibe; cerca del límite, pocas veces.

Fallos, alertas y particiones

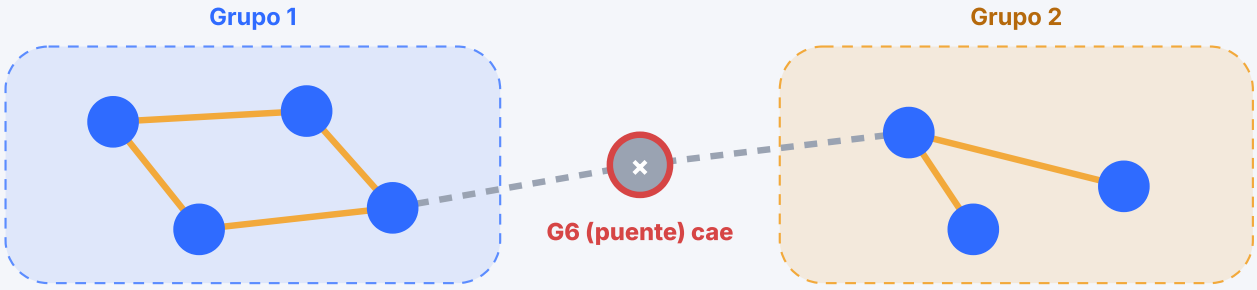
⏱ Timeout y alerta

Cada Gateway revisa cada 5 s a quién lleva mucho sin oír. Si pasan más de **30 s** (`timeout`) sin oír a otro Gateway, lo marca como **caído** y se enciende una **alerta**. Cuando vuelve a oírlo (un OGM nuevo o un beacon), la apaga.

Ojo: "parecer caído" no es "estar caído". Un nodo vivo que quedó fuera de alcance también dispara alertas.

💥 Cómo cae un nodo

- **Batería:** se agota poco a poco.
- **Programado:** el escenario dice "cae G6 a los 60 s".
- **Al azar / dirigido:** cae uno cualquiera, o el más crítico.
- **Manual:** lo tumbas tú (tecla `F` o menú).



Resultado: la malla queda en 2 "componentes" → **red partida**

Partición

La malla de Gateways se separa en grupos que no se pueden hablar. Se cuenta en "componentes": 1 = unida, 2 o más = partida.

Nodo crítico

Un nodo que, si cae, parte la malla (en teoría de grafos, "punto de articulación"). G6 en el dibujo. Cuantos menos haya, más robusta es la red.

Reconvergencia

Lo que tarda la red en volver a estar bien después de un problema: la malla vuelve a unirse y todas las rutas vuelven a estar al día.

Ruta fantasma 👻

El nodo **cree** que tiene ruta hacia otro, pero la radio ya no los conecta. Es información vieja: típico justo después de un fallo.

Ruta faltante

Al revés: la radio **sí** los conecta, pero el nodo todavía no aprendió la ruta. Típico al arrancar o cuando alguien vuelve.

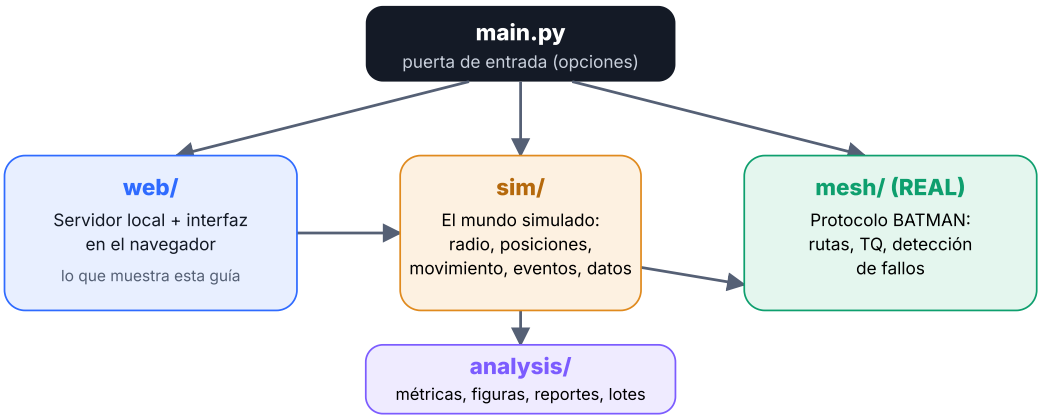
Plano de control vs. plano de datos

Control = los OGM y beacons, que arman las tablas. **Datos** = los mensajes "de verdad" entre usuarios (por ejemplo, N1 le manda algo a N6). Cada paquete de datos avanza salto a salto usando la tabla del nodo que lo tiene en ese momento; si la tabla está mal, el paquete se pierde aunque exista camino por radio.

3 · CÓMO ESTÁ ARMADO

Cuatro carpetas, cuatro trabajos

La regla de oro: lo de `mesh/` es el protocolo real; lo de `sim/` es el mundo inventado alrededor.



Cómo avanza una corrida

La simulación avanza en **pasos de 0.5 s**. En cada paso: `sim/` mueve los nodos y entrega los mensajes (perdiendo algunos según la fiabilidad), `mesh/` decide las rutas dentro de cada nodo y `analysis/` anota todas las medidas. A velocidad 1× se simulan 9 segundos por cada segundo real.

Formas de correrlo

Quiero...	Comando
Verlo en el navegador (lo de esta guía)	`python main.py --web --escenario base`
Verlo en una ventana (pygame)	`python main.py --escenario base`
Correr sin ventana y obtener un resumen	`python main.py --headless --escenario base --seed 1`
Ver el estado interno del protocolo	`python main.py --inspect --escenario particion --duracion 30`
Repetir con muchas semillas y promediar	`python main.py --batch lotes/robustez.json`

En Windows, con el entorno virtual: `venv\Scripts\python main.py …`. Opciones útiles: `--seed N` (corrida repetible), `--static` (nadie se mueve), `--movilidad descentralizada|seguir|repartir`, `--trafico K` (K flujos de datos).

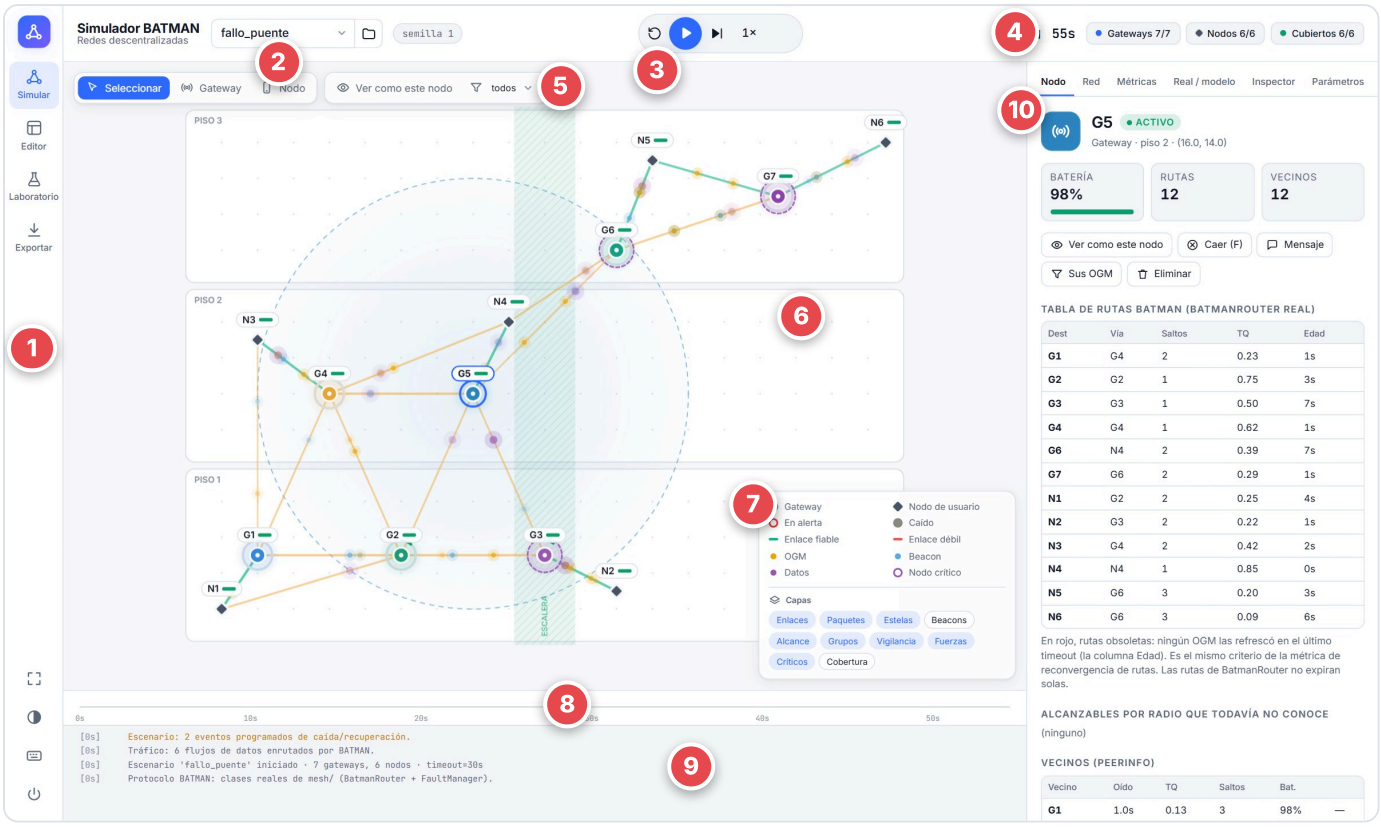
Los escenarios que trae

Escenario	Qué plantea
base	4 Gateways juntos arriba y 3 usuarios abajo: el caso de referencia.
colapso_progresivo	Baterías que se gastan rápido: los Gateways caen uno tras otro.
particion	Alcance corto y dos parejas en esquinas opuestas: la malla arranca partida.
rescatista_perdido	Un Gateway se aleja y pierde el contacto.
denso	8 Gateways y 6 usuarios: muchos radios.
campo_abierto	Terreno de 120 × 80 m, usuarios que caminan en grupos, 6 flujos de datos.
fallos/...	Los experimentos del Capítulo 5: 7 Gateways fijos en 3 pisos con distintos fallos (puente, borde, cascada...).

4 · LA PANTALLA PRINCIPAL

Un vistazo a la interfaz web

Se abre con `python main.py --web`. Captura real: escenario **fallo_puente**, a los 55 s, con el Gateway G5 seleccionado.

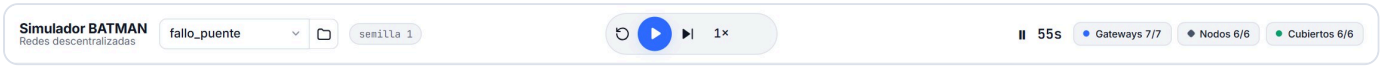


1. **Barra lateral (riel):** Simular, Editor de escenarios, Laboratorio y Exportar. Abajo: modo presentación, tema claro/oscuro, atajos y terminar.
2. **Escenario y semilla:** qué red está cargada y con qué semilla. El ícono de carpeta abre “Elegir la red”.
3. **Reproducción:** reiniciar, play/pausa, avanzar un paso de 0.5 s y la velocidad.
4. **Reloj y contadores:** tiempo simulado, Gateways vivos, Nodos vivos y usuarios cubiertos. Aquí aparecen los avisos (red partida, intervenciones).
5. **Herramientas del mapa:** seleccionar, agregar Gateway/Nodo, “Ver como este nodo” y filtro de OGM.
6. **El mapa:** el edificio de 3 pisos visto de frente (corte), con los nodos, los enlaces y los paquetes en vuelo.
7. **Leyenda y capas:** qué significa cada símbolo y botones para prender o apagar capas de información.
8. **Línea de tiempo:** un ícono por cada evento importante (caídas, alertas, particiones…).
9. **Log:** el registro de texto de lo que va pasando, con su instante.
10. **Panel lateral:** seis pestañas (Nodo, Red, Métricas, Real/modelo, Inspector, Parámetros).

4 · LA PANTALLA PRINCIPAL

Las barras, de cerca

Barra superior



Elemento	Qué hace
Selector de escenario y carpeta	Cambia la red. La carpeta abre un diálogo con los escenarios, archivos propios o una red aleatoria.
semilla 1	El número que fija el azar. Misma semilla = misma corrida. Clic para cambiarla.
↺ **Reiniciar**	Vuelve a empezar con la misma red y la misma semilla (`R`).
▶ / ⏸	Pausa o reanuda (`Espacio`).
⏭ **Un paso**	Avanza 0.5 s; sólo funciona en pausa. Ideal para ver un evento con calma.
1×	Velocidad: de 0.25× a 8× o “máx.”. 1× = 9 s simulados por segundo real.
⏸ **55s**	Reloj de la simulación (no es el reloj de tu computador).
Gateways 7/7 · Nodos 6/6	Cuántos están vivos de cuántos hay.
Cubiertos 6/6	Usuarios que comparten grupo de radio con algún Gateway (tienen cobertura).
RED PARTIDA (2) · 1 intervención	Aparecen sólo cuando aplica: la malla está en 2 pedazos; tú tocaste la corrida (ya no equivale a un comando de terminal).



Barra lateral (riel)

- **Simular:** la pantalla principal.
- **Editor:** diseñar un escenario propio (sección 11).
- **Laboratorio:** correr experimentos con muchas semillas (sección 11).
- **Exportar** (`P`): guarda la figura de análisis y los reportes en `reportes/` .
- Abajo: **modo presentación** (`Z` , oculta lo accesorio para proyectar), **tema claro/oscuro**, **atajos de teclado** y **terminar** (exporta y apaga el servidor).

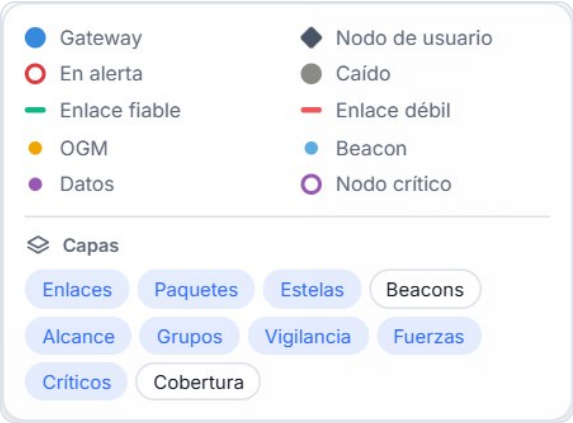
Línea de tiempo y log



Arriba, un ícono por evento sobre el eje de tiempo (morado = partición, naranja = alerta, verde = recuperación). Abajo, el log con el instante simulado entre corchetes.

5 · EL MAPA Y SUS CAPAS

Cómo leer el mapa



Leyenda y botones de capas (los azules están encendidos).

- **Pisos:** el edificio se ve de frente: piso 1 abajo, piso 3 arriba. La franja verde rayada es la **escalera**.
- **Círculo de color** = Gateway (cada uno con su color); **rombo oscuro** = Nodo de usuario; **gris** = caído; **anillo rojo** = en alerta (alguien cree que está caído).
- **Etiqueta** (G5, N3): con una barrita de batería al lado (verde = llena).
- **Líneas:** enlaces de radio. Verde = fiable, naranja/rojo = débil (se pierden más paquetes).
- **Puntitos que viajan:** naranja = OGM, celeste = beacon, morado = datos.
- **Anillo morado punteado** = nodo crítico.

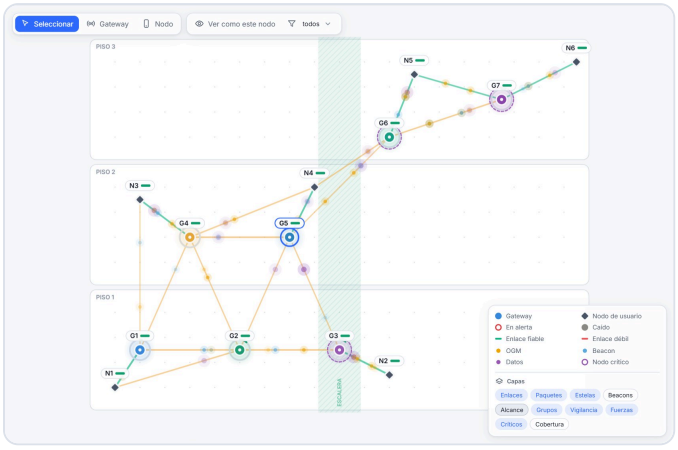
Ratón: clic selecciona, arrastrar mueve un nodo, pasar por encima muestra un recuadro con datos, clic derecho abre un menú.

Capas: información que se prende y se apaga

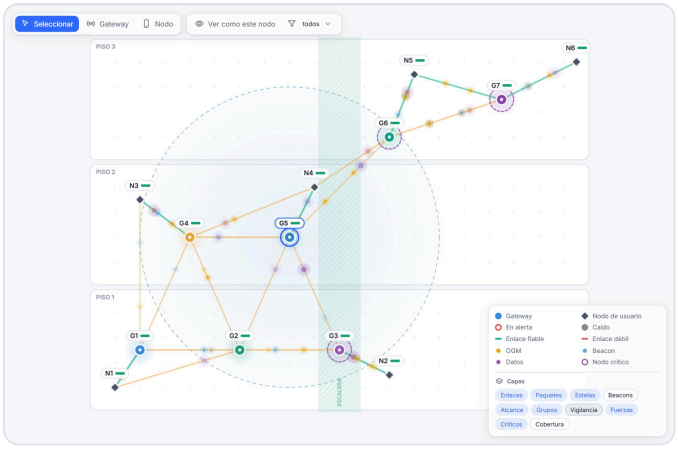
Capa	Qué muestra
Enlaces	Las líneas de radio entre nodos, coloreadas por fiabilidad.
Paquetes	Los mensajes en vuelo (OGM, beacons, datos).
Estelas	El rastro de los últimos movimientos de cada nodo.
Beacons	Un pulso alrededor del nodo cada vez que emite su “hola”.
Alcance	El círculo hasta donde llega la radio del nodo seleccionado.
Grupos	Una mancha de color por cada pedazo de la malla cuando está partida.
Vigilancia	A quién vigila el Gateway seleccionado y cuánto tiempo lleva sin oírlo.
Fuerzas	Con Gateways móviles: hacia dónde decide moverse cada uno y por qué.
Críticos	Marca los nodos que, si caen, parten la malla.
Cobertura (`C`)	Un “mapa de calor” de la señal: qué tan buena es la conexión a algún Gateway en cada punto.

5 · EL MAPA Y SUS CAPAS

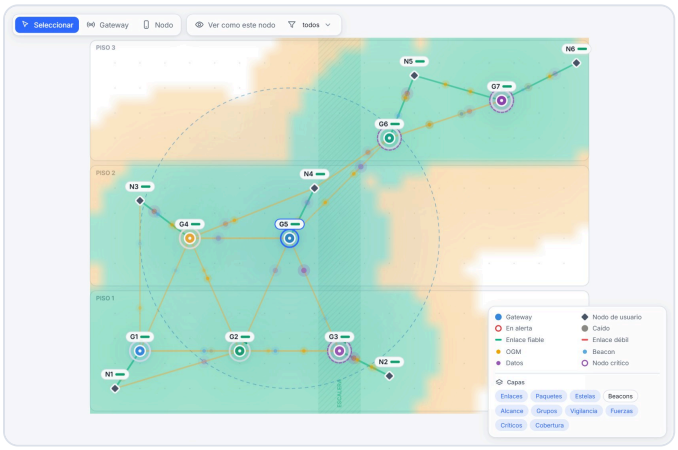
Las capas, en imágenes



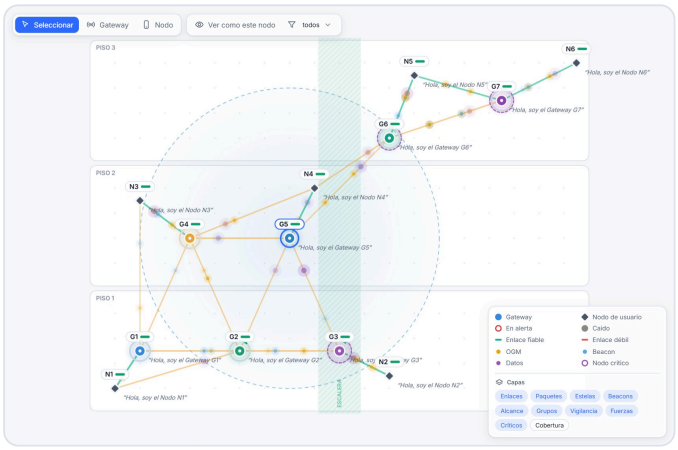
Alcance. El círculo punteado es hasta dónde llega la radio de G5 (12 m en este escenario). Fuera de él, la fiabilidad es 0.



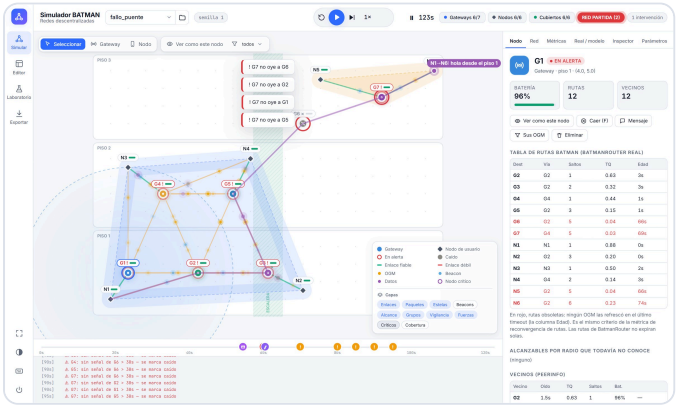
Vigilancia. Los anillos alrededor de los Gateways que G5 vigila: se van llenando con el silencio y, al pasar el timeout (30 s), los da por caídos.



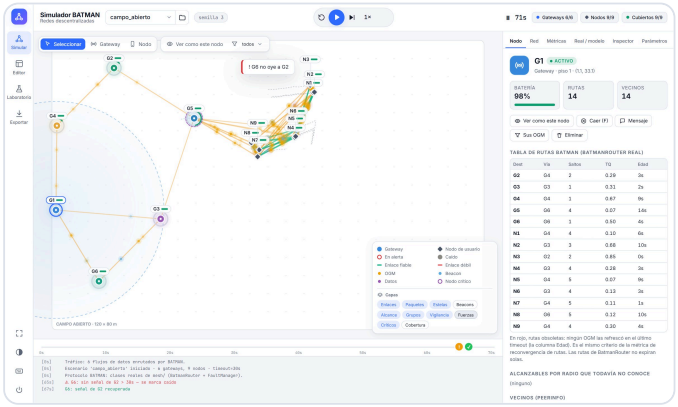
Cobertura. Cuanto más intenso el color, mejor la señal hacia algún Gateway vivo. Las zonas en blanco no tienen cobertura: ahí un usuario quedaría desconectado.



Beacons. Burbujas con el "hola" de cada nodo: muestran quién se está anunciando a sus vecinos en este momento.



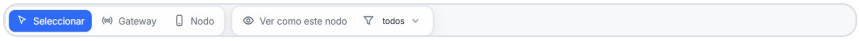
Grupos y Críticos (a los 123 s, con G6 caído). La mancha azul es un pedazo de la malla y la naranja el otro: la red está partida. Los anillos rojos son las alertas.



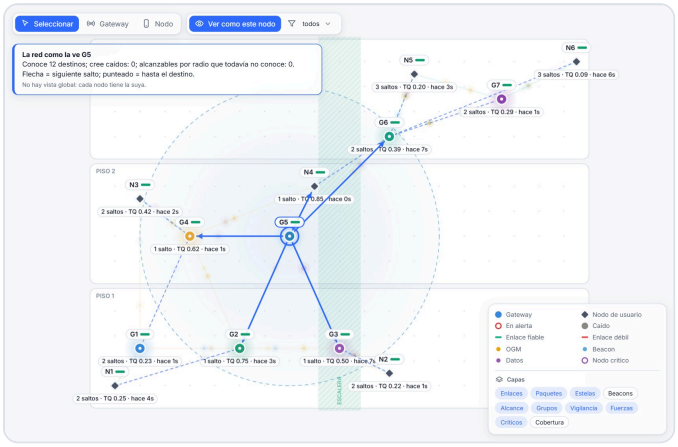
Fuerzas, en *campo_abierto* (vista en planta, 120 × 80 m). Los Gateways salen juntos y deciden solos, con lo que oyen, hacia dónde moverse: cubrir usuarios, separarse o reconectar.

6 · HERRAMIENTAS DEL MAPA

Ver, filtrar y mandar mensajes

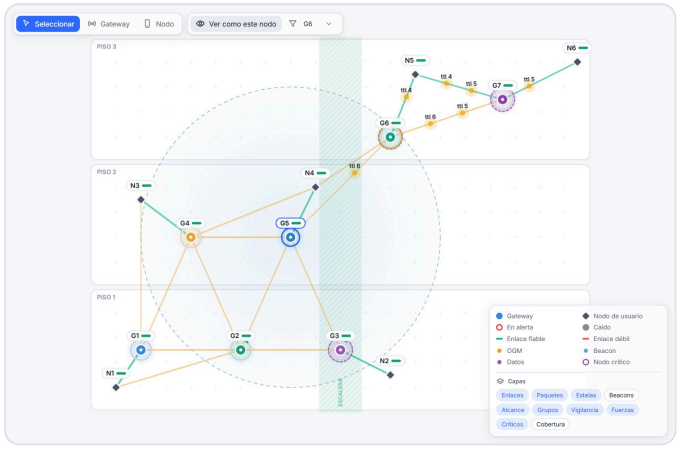


Barra de herramientas sobre el mapa.



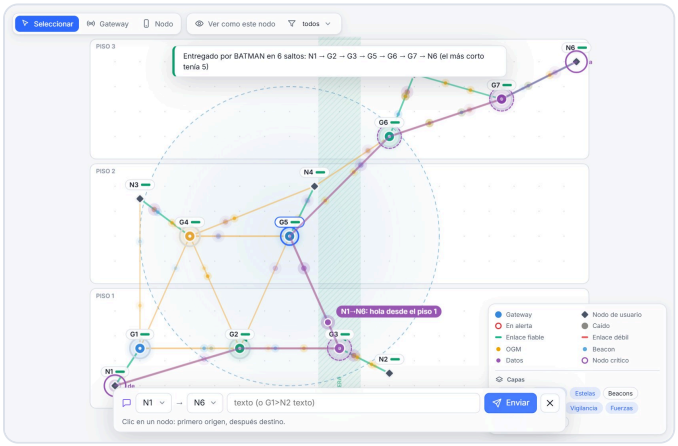
👁 Ver como este nodo (L)

Muestra la red **sólo como la conoce el nodo seleccionado** (aquí G5). Las flechas son su siguiente salto hacia cada destino y cada etiqueta dice a cuántos saltos cree que está, el TQ y hace cuánto lo oyó. Es la prueba visual de que **no hay vista global**: cada nodo tiene la suya.



Filtro “OGM de” (O)

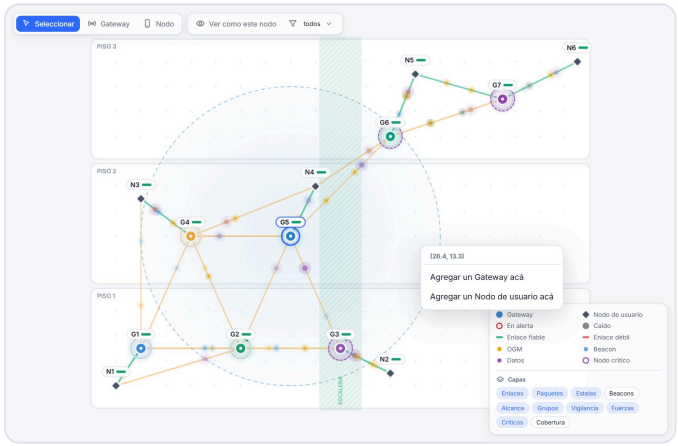
Deja ver sólo los OGM que originó un nodo (aquí G6), con su TTL bajando en cada salto. Sirve para ver cómo se “inunda” la malla con el “aquí estoy” de un nodo.



✉ Mandar un mensaje (M)

Eliges origen y destino (o haces clic en dos nodos) y escribes un texto. El mensaje viaja **salto a salto por las tablas de BATMAN**. La interfaz dice el camino real y lo compara con el más corto posible.

> Ejemplo real: *“Entregado por BATMAN en 6 saltos: N1 → G2 → G3 → G5 → G6 → G7 → N6 (el más corto tenía 5)”*. Llegó, pero dio una vuelta de más: eso es el **estiramiento** (6/5 = 1.2).



Clic derecho

En un lugar vacío: agregar un Gateway o un Nodo de usuario ahí. Sobre un nodo: ver como él, caer/recuperar, mensaje desde/hasta él, mostrar sus OGM o eliminarlo.

También: herramientas **Gateway** y **Nodo** de la barra (clic en el mapa = nodo nuevo), arrastrar para mover, y teclas F / G para tumbar/recuperar el seleccionado.

7 · EL PANEL LATERAL

Pestaña “Nodo”: la mente de un nodo



Al seleccionar un nodo, muestra **su estado interno real**, leído directamente del `BatmanRouter` y del `FaultManager` que corren dentro de él.

Cabecera

Nombre, estado (**activo**, **en alerta** o **inactivo**), rol, piso y posición. Tres datos: **batería**, cuántas **rutas** conoce y cuántos **vecinos** tiene.

Botones

Ver como este nodo · Caer / Recuperar · Mensaje · Sus OGM · Eliminar.

Tabla de rutas BATMAN

Una fila por destino:

- **Dest:** a quién quiere llegar.
- **Vía:** a qué vecino se lo pasa (siguiente salto).
- **Saltos:** cuántos saltos cree que hay.
- **TQ:** la calidad de esa ruta (0 a 1).
- **Edad:** hace cuánto un OGM refrescó esa ruta.

En **rojo**, las rutas **obsoletas**: nadie las refrescó en los últimos 30 s. Las rutas no se borran solas, por eso la interfaz las marca.

Más abajo

- **Alcanzables por radio que todavía no conoce:** nodos que la radio ya conecta pero aún no tienen ruta.
- **Vecinos (PeerInfo):** a quién oye directo, hace cuánto, TQ, batería.
- **Detección de fallos** (sólo Gateways): una barra por cada Gateway vigilado que se llena con el silencio hasta el timeout.
- **Cree caídos:** a quién da por muerto este nodo.

7 · EL PANEL LATERAL

Pestaña “Nodo” durante un fallo



Nodo | Red | Métricas | Real / modelo | Inspector | Parámetros

G1 ● EN ALERTA
Gateway · piso 1 · (4.0, 5.0)

BATERÍA 96% | RUTAS 12 | VECINOS 12

Ver como este nodo | Caer (F) | Mensaje | Sus OGM | Eliminar

TABLA DE RUTAS BATMAN (BATMANROUTER REAL)

Dest	Vía	Saltos	TQ	Edad
G2	G2	1	0.63	3s
G3	G2	2	0.32	3s
G4	G4	1	0.44	1s
G5	G2	3	0.15	1s
G6	G2	5	0.04	66s
G7	G4	5	0.03	69s
N1	N1	1	0.88	0s
N2	G2	3	0.20	0s
N3	N3	1	0.50	2s
N4	G4	2	0.14	3s
N5	G2	5	0.04	66s
N6	G2	6	0.23	74s

En rojo, rutas obsoletas: ningún OGM las refrescó en el último timeout (la columna Edad). Es el mismo criterio de la métrica de reconvergencia de rutas. Las rutas de BatmanRouter no expiran solas.

ALCANZABLES POR RADIO QUE TODAVÍA NO CONOCE

(ninguno)

VECINOS (PEERINFO)

Vecino	Oído	TQ	Saltos	Bat.	
G2	1.5s	0.63	1	96%	—

La misma pestaña, ahora con **G1** a los 123 s, cuando G6 (el puente al piso 3) lleva 60 s caído.

Qué se ve

- G1 aparece **EN ALERTA**: otros Gateways (G7, del otro lado) dejaron de oírlo.
- Las rutas a **G6, G7, N5 y N6** están en **rojo**: edad de 66–74 s, TQ muy bajo (0.03–0.23). Son **rutas fantasma**: G1 todavía las tiene, pero ya no sirven.
- Las rutas del lado de G1 (G2, G3, G4…) siguen frescas (edad de 0–3 s).

Esto es justamente lo que la tesis quiere mostrar: tras un fallo, cada nodo **se entera por su cuenta y con retraso**, y durante un rato cree cosas que ya no son verdad.

7 · EL PANEL LATERAL

Pestaña “Red”: la matriz de conocimiento



A los 55 s: todo en verde.



A los 123 s, con G6 caído.

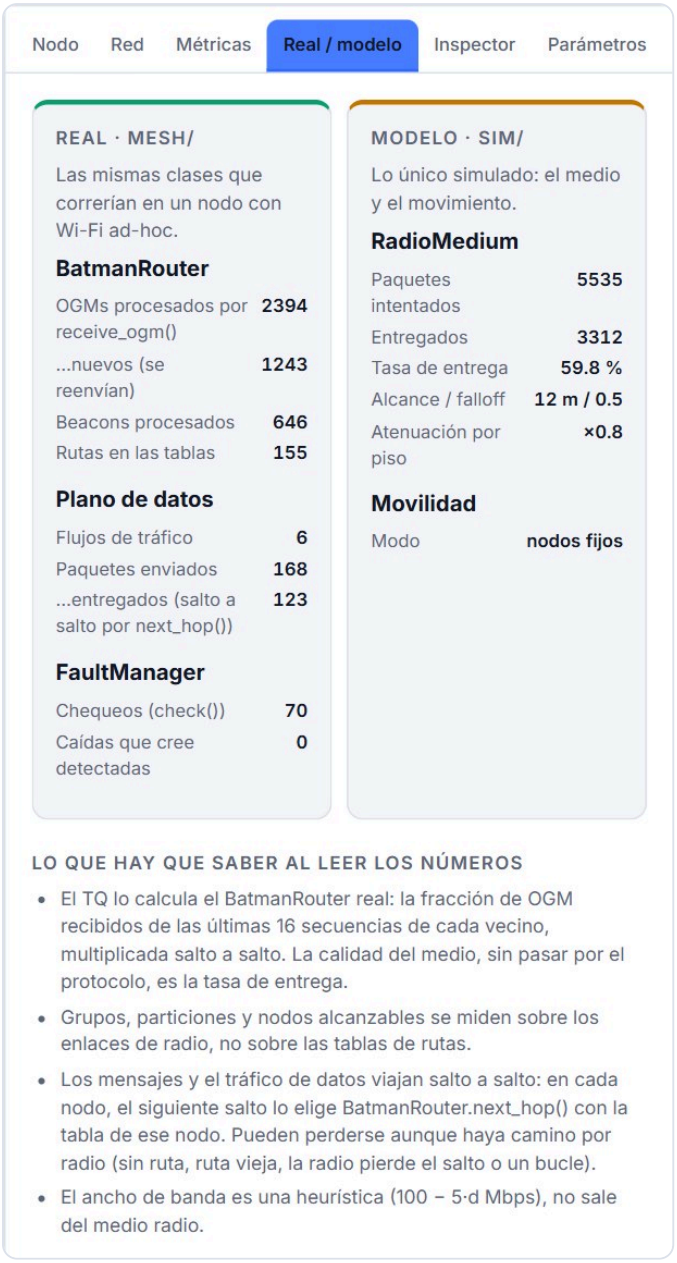
Una cuadrícula donde **la fila es el nodo que sabe** y **la columna es el destino**. Cada casilla responde: “¿este nodo sabe llegar a ese otro?”. El número es cuántos saltos.

Color	Significa
■ Verde con número	Ruta vigente, con los saltos.
■ Rojo	Ruta obsoleta: hace más de 30 s que nadie la refresca.
■ Naranja/marrón	La radio los conecta, pero todavía no hay ruta (falta aprender).
■ Gris claro	Sin conexión física (o la diagonal: uno mismo).
Borde punteado	Tiene la ruta, pero la radio ya no los conecta (ruta fantasma).

> La barra de arriba resume la convergencia: **“154 de 156 pares conectados por radio ya tienen una ruta vigente (99 %)”**. Cuando cae un nodo, baja; cuando la red se recupera, vuelve a acercarse al 100 %.

7 · EL PANEL LATERAL

“Real / modelo” y “Parámetros”



Real / modelo

Deja claro **qué parte es código real del protocolo y qué parte es simulación**. Es el argumento central de la tesis en una sola pantalla.

- **Real (mesh/):** cuántos OGM procesó el `BatmanRouter`, cuántos eran nuevos, beacons, rutas en tablas; paquetes de datos enviados y entregados; chequeos del `FaultManager`.
- **Modelo (sim/):** paquetes que el medio de radio intentó y entregó (aquí 59.8 %), alcance, falloff, atenuación por piso y el modo de movilidad.

Al final recuerda cómo leer los números: el TQ sale del protocolo; los grupos y particiones se miden con la radio; el ancho de banda es una estimación.

Inspector (`I`)

El mismo texto que da `--inspect` en la terminal: el estado interno de todos los nodos de una vez, para copiarlo o revisarlo con detalle.

7 · EL PANEL LATERAL

Pestaña “Parámetros”: cambiar las reglas en vivo



Deslizadores que cambian la simulación **mientras corre**. Cada uno muestra el valor original y un botón **restaurar**.

Parámetro	Qué controla
Medio radio	
Alcance (m)	Hasta dónde llega la señal.
Degradación (falloff)	Cuánto empeora con la distancia.
Pérdida base	Paquetes que se pierden aunque estén pegados.
Atenuación por piso	Cuánto queda de la señal al cruzar un piso.
Protocolo	
Timeout (s)	Silencio para dar a alguien por caído.
TTL de OGM	Cuántos saltos viaja un OGM.
Beacon / OGM cada (s)	Cada cuánto se anuncia cada nodo.
Batería y movilidad	
Drenaje	Gasto de batería por segundo.
Velocidad / Movilidad	Si los Gateways se mueven y cómo: *descentralizada* (cada uno decide con lo que oye), *seguir* o *repartir* (centralizadas, para comparar).
Movilidad de usuarios	Quietos, random waypoint o en grupos.
Plano de datos	
Protocolo de datos	BATMAN o inundación (el método simple, para comparar).

Atajos: `+` / `-` alcance, `[` / `]` falloff.

8 · HISTORIA DE UN FALLO

Qué pasa cuando cae el puente

Escenario **fallo_puente** (semilla 1, 6 flujos de datos): 7 Gateways fijos en 3 pisos. G6 es el único puente hacia el piso 3. Cae a los 60 s y vuelve a los 150 s.

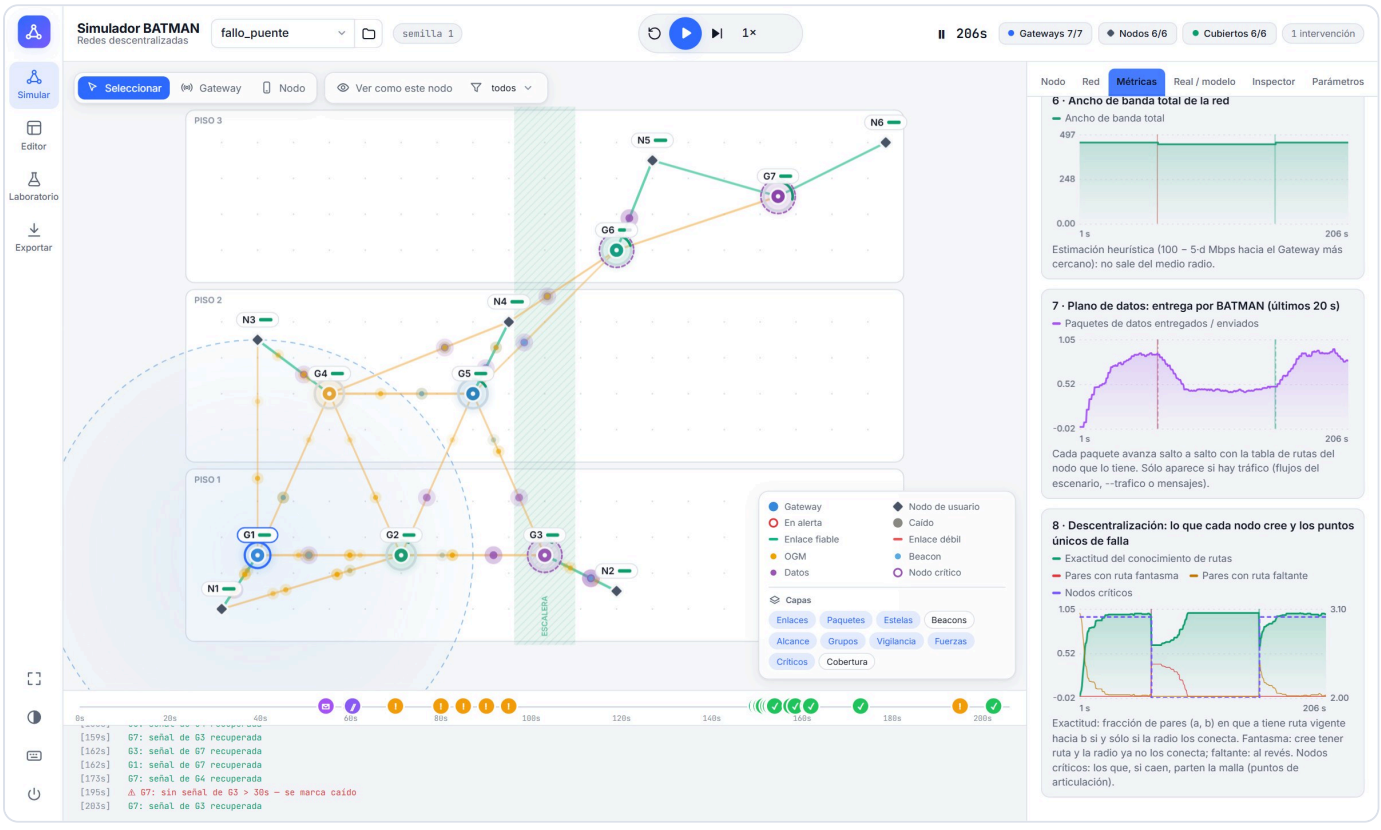
0–60 s **Todo bien.** Los OGM arman las tablas; la matriz llega al 99 %. Los datos se entregan cada vez mejor (los primeros segundos se pierden mientras las rutas se arman).

60 s **Cae G6.** En el mismo instante la malla queda en dos pedazos: aparece **RED PARTIDA (2)**. Pero **nadie lo sabe todavía**: los nodos sólo notan silencio.

60–90 s **Silencio creciente.** En la capa Vigilancia los anillos se van llenando. Las tablas aún tienen rutas por G6: los datos que van al piso 3 se pierden.

≈ 90 s **Alertas.** Pasado el timeout de 30 s (más hasta 5 s del chequeo), cada Gateway decide por su cuenta: “G7 no oye a G1”, “G4: sin señal de G6” … se encienden los anillos rojos.

150 s **Vuelve G6.** La malla se reunifica, llegan OGM nuevos que apagan las alertas y las rutas se rehacen. La entrega de datos sube de nuevo.



A los 206 s: G6 ya volvió. En la línea de tiempo de abajo se ve toda la historia: partición (morado), alertas (naranja) y recuperaciones (verde).

9 · LAS 8 GRÁFICAS EN VIVO

Pestaña “Métricas”

Las mismas gráficas que se exportan en `analisis_red.png`, pero en vivo. Todas son de la corrida de la sección 8. Las rayas verticales marcan eventos: **roja** = cae un nodo, **verde** = vuelve, **morada** = partición / reunificación.



1 · Nodos activos

Qué mide: cuántos Gateways (azul) y usuarios (rojo) están vivos.

Cómo leerla: un escalón hacia abajo = alguien cayó; hacia arriba = volvió.

En el ejemplo: los Gateways bajan de 7 a 6 a los 60 s y vuelven a 7 a los 150 s.

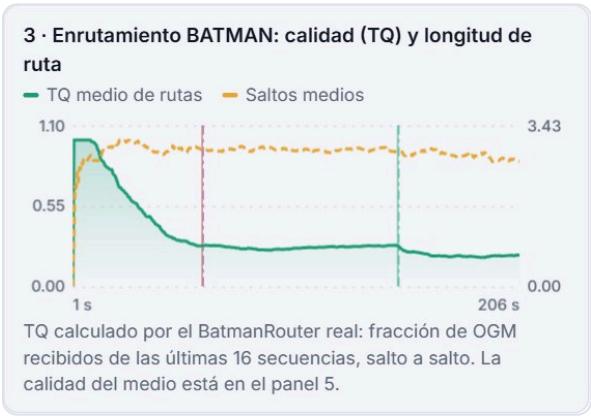


2 · Auto-reorganización de la malla

Qué mide: en cuántos pedazos está la malla (morado) y cuántos usuarios tienen cobertura (rojo).

Cómo leerla: 1 = unida (la línea verde de referencia); 2 o más = partida, y esa zona se sombrea.

En el ejemplo: sube a 2 entre los 60 y 150 s. Los usuarios siguen cubiertos porque cada pedazo conserva algún Gateway.



TQ calculado por el BatmanRouter real: fracción de OGM recibidos de las últimas 16 secuencias, salto a salto. La calidad del medio está en el panel 5.

3 · Calidad (TQ) y largo de las rutas

Qué mide: el TQ promedio de todas las rutas (verde) y cuántos saltos tienen en promedio (naranja punteado, eje derecho).

Cómo leerla: TQ más alto = rutas más confiables. Empieza en 1 porque al arrancar todavía no hay historial de pérdidas; baja hasta su valor real.

En el ejemplo se estabiliza alrededor de 0.3: con enlaces de 12 m entre pisos se pierden bastantes OGM.



4 · Detección de Gateway perdido

Qué mide: el mayor silencio (en segundos) que algún Gateway lleva sin oír a otro.

Cómo leerla: la raya punteada es el timeout (30 s). Cuando la curva la cruza, se dispara una alerta y la zona se sombrea de naranja.

En el ejemplo: la curva sube en rampa desde los 60 s (nadie oye a G6) hasta que G6 vuelve a los 150 s y cae de golpe.

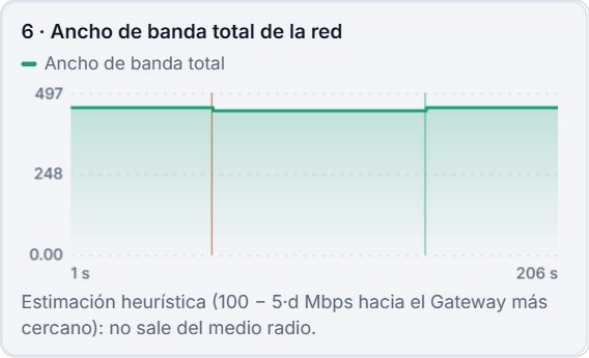


5 · Calidad del medio radio

Qué mide: qué fracción de los paquetes transmitidos llegó (gris = cada paso, azul = promedio móvil).

Cómo leerla: es la calidad de la radio "pura", sin pasar por el protocolo. 1 = todo llega.

En el ejemplo ronda el 0.6: se pierden 4 de cada 10 transmisiones.



6 · Ancho de banda total

Qué mide: una estimación del ancho de banda de la red.

Cómo leerla: sirve para ver tendencias (baja si caen nodos). **Es una heurística** (100 − 5 × distancia Mbps al Gateway más cercano), no sale del medio radio; así lo dice la propia gráfica.

9 · LAS 8 GRÁFICAS EN VIVO



Cada paquete avanza salto a salto con la tabla de rutas del nodo que lo tiene. Sólo aparece si hay tráfico (flujos del escenario, --trafico o mensajes).

7 · Plano de datos: entrega

Qué mide: de los paquetes de datos enviados en los últimos 20 s, qué fracción llegó a su destino.

Cómo leerla: es la métrica que más le importa a un usuario. Sólo aparece si hay tráfico.

En el ejemplo: llega a ~0.85, cae a ~0.45 mientras G6 está caído (los datos hacia el piso 3 no tienen camino) y se recupera cuando vuelve.



Exactitud: fracción de pares (a, b) en que a tiene ruta vigente hacia b si y sólo si la radio los conecta. Fantasma: cree tener ruta y la radio ya no los conecta; faltante: al revés. Nodos críticos: los que, si caen, parten la malla (puntos de articulación).

8 · Descentralización

Qué mide:

- **Exactitud** (verde): de todas las parejas de nodos, en qué fracción lo que el nodo *cree* coincide con la realidad.
- **Rutas fantasma** (rojo) y **faltantes** (marrón): los dos tipos de error.
- **Nodos críticos** (morado punteado, eje derecho).

En el ejemplo: al caer G6 la exactitud baja de golpe y suben las fantasma (creen rutas que ya no existen); poco a poco se corrigen. Al volver G6 suben las faltantes (falta aprender las rutas nuevas).

Arriba a la derecha

Un selector cambia la ventana de tiempo: últimos 60 s, últimos 200 s o toda la corrida. Pasando el ratón por una gráfica sale el valor exacto.

10 · LAS MÉTRICAS DEL RESUMEN

Las 28 métricas, una por una

Al terminar una corrida (o un lote), el simulador resume todo en números. Son los que van a las tablas del Capítulo 5. La columna “Mejor” dice hacia dónde conviene que vayan.

Calidad de las rutas y del radio

Métrica	En palabras simples	Mejor
TQ medio de rutas	Nota promedio de calidad de todas las rutas (0 a 1), promediada en el tiempo.	↑ alto
Saltos medios por ruta	Cuántos “pases” hay en promedio entre dos nodos.	↓ bajo
Tasa de entrega del radio	De todo lo que se transmitió por radio, qué fracción llegó.	↑ alto

Conectividad y particiones

Métrica	En palabras simples	Mejor
Componentes de la malla al final	En cuántos pedazos terminó la malla. 1 = unida.	= 1
Usuarios alcanzables al final	Cuántos usuarios tenían cobertura al terminar.	↑ alto
Gateways vivos al final	Cuántos Gateways sobrevivieron.	↑ alto
Episodios de partición	Cuántas veces se partió la malla.	↓ bajo
Tiempo con la malla partida (s)	Cuántos segundos pasó partida en total.	↓ bajo
Tiempo de reconvergencia (s)	Cuánto duró en promedio una partición que se volvió a unir (por radio). “No aplica” si nunca se reunificó.	↓ bajo
Cobertura media de usuarios	Promedio en el tiempo de la fracción de usuarios con cobertura. A diferencia de “al final”, sí cuenta lo que se perdió mientras un Gateway estuvo caído.	↑ alto

Detección de fallos

Métrica	En palabras simples	Mejor
Alertas de Gateway perdido	Cuántas veces un Gateway dio por caído a otro. Incluye falsas alarmas (enlaces que parpadean).	depende
Primera alerta (s)	En qué segundo saltó la primera alerta. Comparado con el momento del fallo, dice qué tan rápido se detectó.	depende
Reconvergencia de rutas BATMAN (s)	Desde que vuelve un Gateway (o se reúne la malla) hasta que todos los pares conectados vuelven a tener ruta vigente. Es lo que tarda *el protocolo*, no la radio.	↓ bajo

10 · LAS MÉTRICAS DEL RESUMEN

Plano de datos (sólo si hay tráfico)

Métrica	En palabras simples	Mejor
Entrega de datos	De los mensajes que se mandaron entre usuarios, qué fracción llegó.	↑ alto
Entrega con camino por radio	Lo mismo, pero contando sólo los mensajes que *podían* llegar (había camino físico). Mide qué tan bien enruta el protocolo, sin culparlo de lo imposible. **La métrica clave para comparar protocolos.**	↑ alto
Saltos de los datos entregados	Cuántos saltos dieron en promedio los mensajes que llegaron.	↓ bajo
Estiramiento	Saltos reales ÷ saltos del camino más corto. 1.0 = eligió el mejor camino; 1.2 = 20 % de vuelta extra.	→ 1.0
Transmisiones por paquete	Cuánto "cuesta" entregar un mensaje (intentos de envío). Sirve para comparar BATMAN con la inundación.	↓ bajo

¿Por qué se pierde un paquete de datos? (las 6 causas suman 1 – entrega)

Causa	Qué pasó	Culpa de...
Destino caído	El destinatario estaba apagado.	la física
Sin camino por radio	No había forma física de llegar (red partida o usuario sin cobertura).	la física
Sin ruta	Algún nodo del camino no tenía ruta hacia el destino (todavía no la aprendía).	el protocolo
Ruta vieja	La tabla mandó el paquete a un vecino que ya estaba caído o lejos.	el protocolo
La radio perdió el salto	El salto falló aun después de 3 reintentos.	el medio
Bucle	El paquete empezó a dar vueltas entre nodos.	el protocolo

Descentralización

Métrica	En palabras simples	Mejor
Exactitud del conocimiento	Promedio de "qué tanto acierta cada nodo sobre a quién puede llegar". 1 = todos saben exactamente la verdad.	↑ alto
Pares con ruta fantasma	Fracción de parejas en que el nodo cree tener ruta que ya no existe.	↓ bajo
Pares con ruta faltante	Fracción de parejas que podrían hablarse pero el nodo aún no lo sabe.	↓ bajo
Nodos críticos (media)	Cuántos puntos únicos de falla tuvo la malla en promedio.	↓ bajo

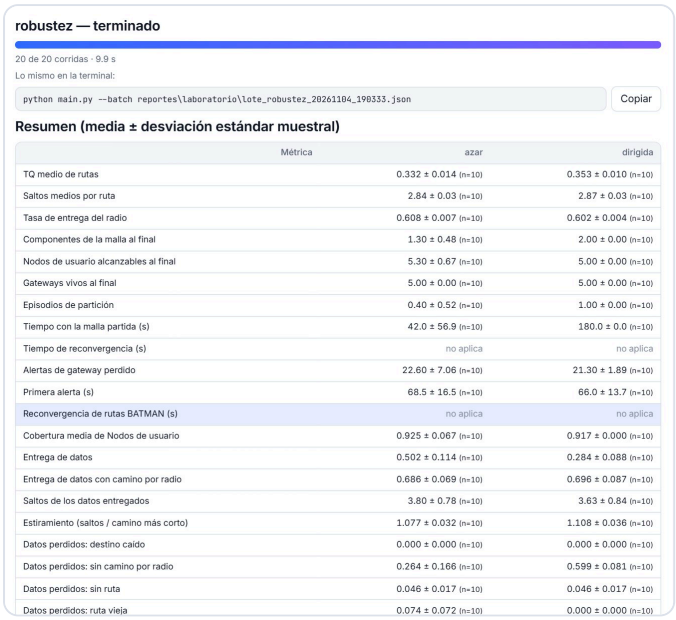
Media ± desviación

En los lotes cada métrica sale como **media ± desviación estándar** sobre las semillas, por ejemplo *0.502 ± 0.114 (n=10)*: en promedio 0.502, con una variación típica de 0.114 entre corridas, sobre 10 corridas.

11 · LABORATORIO DE EXPERIMENTOS

De jugar con la red a tener resultados

El laboratorio corre un **lote**: varios escenarios × varias semillas, igual que `python main.py --batch`, y muestra la tabla de resultados. Da exactamente los mismos números que la terminal.



robustez — terminado

20 de 20 corridas · 9.9 s

Lo mismo en la terminal:

```
python main.py --batch reportes\laboratorio\lote_robustez_20261104_190333.json
```

Copiar

Resumen (media ± desviación estándar muestral)

Métrica	azar	dirigida
TQ medio de rutas	0.332 ± 0.014 (n=10)	0.353 ± 0.010 (n=10)
Saltos medios por ruta	2.84 ± 0.03 (n=10)	2.87 ± 0.03 (n=10)
Tasa de entrega del radio	0.608 ± 0.007 (n=10)	0.602 ± 0.004 (n=10)
Componentes de la malla al final	1.30 ± 0.48 (n=10)	2.00 ± 0.00 (n=10)
Nodos de usuario alcanzables al final	5.30 ± 0.67 (n=10)	5.00 ± 0.00 (n=10)
Gateways vivos al final	5.00 ± 0.00 (n=10)	5.00 ± 0.00 (n=10)
Episodios de partición	0.40 ± 0.52 (n=10)	1.00 ± 0.00 (n=10)
Tiempo con la malla partida (s)	42.0 ± 56.9 (n=10)	180.0 ± 0.0 (n=10)
Tiempo de reconvergencia (s)	no aplica	no aplica
Alertas de gateway perdido	22.60 ± 7.06 (n=10)	21.30 ± 1.89 (n=10)
Primera alerta (s)	68.5 ± 16.5 (n=10)	66.0 ± 13.7 (n=10)
Reconvergencia de rutas BATMAN (s)	no aplica	no aplica
Cobertura media de Nodos de usuario	0.925 ± 0.067 (n=10)	0.917 ± 0.000 (n=10)
Entrega de datos	0.502 ± 0.114 (n=10)	0.284 ± 0.088 (n=10)
Entrega de datos con camino por radio	0.686 ± 0.069 (n=10)	0.696 ± 0.087 (n=10)
Saltos de los datos entregados	3.80 ± 0.78 (n=10)	3.63 ± 0.84 (n=10)
Estiramiento (saltos / camino más corto)	1.077 ± 0.032 (n=10)	1.108 ± 0.036 (n=10)
Datos perdidos: destino caído	0.000 ± 0.000 (n=10)	0.000 ± 0.000 (n=10)
Datos perdidos: sin camino por radio	0.264 ± 0.166 (n=10)	0.599 ± 0.081 (n=10)
Datos perdidos: sin ruta	0.046 ± 0.017 (n=10)	0.046 ± 0.017 (n=10)
Datos perdidos: ruta vieja	0.074 ± 0.072 (n=10)	0.000 ± 0.000 (n=10)

Resultado real de `lotes/robustez.json`: 20 corridas de 240 s, en 9.9 s.

Cómo se usa

1. Cargar un lote de `lotes/` o armar las entradas a mano (escenario, etiqueta, nodos fijos, movilidad, flujos de datos).
2. Elegir duración y número de semillas.
3. ▶ **Correr**. Se ve el progreso y el comando equivalente de terminal.
4. Al terminar, clic en una métrica para graficarla.

El experimento del ejemplo

El mismo despliegue sufre **dos caídas** (a los 60 y 120 s). En **azar** caen nodos cualquiera; en **dirigida** cae siempre el nodo **más crítico**.

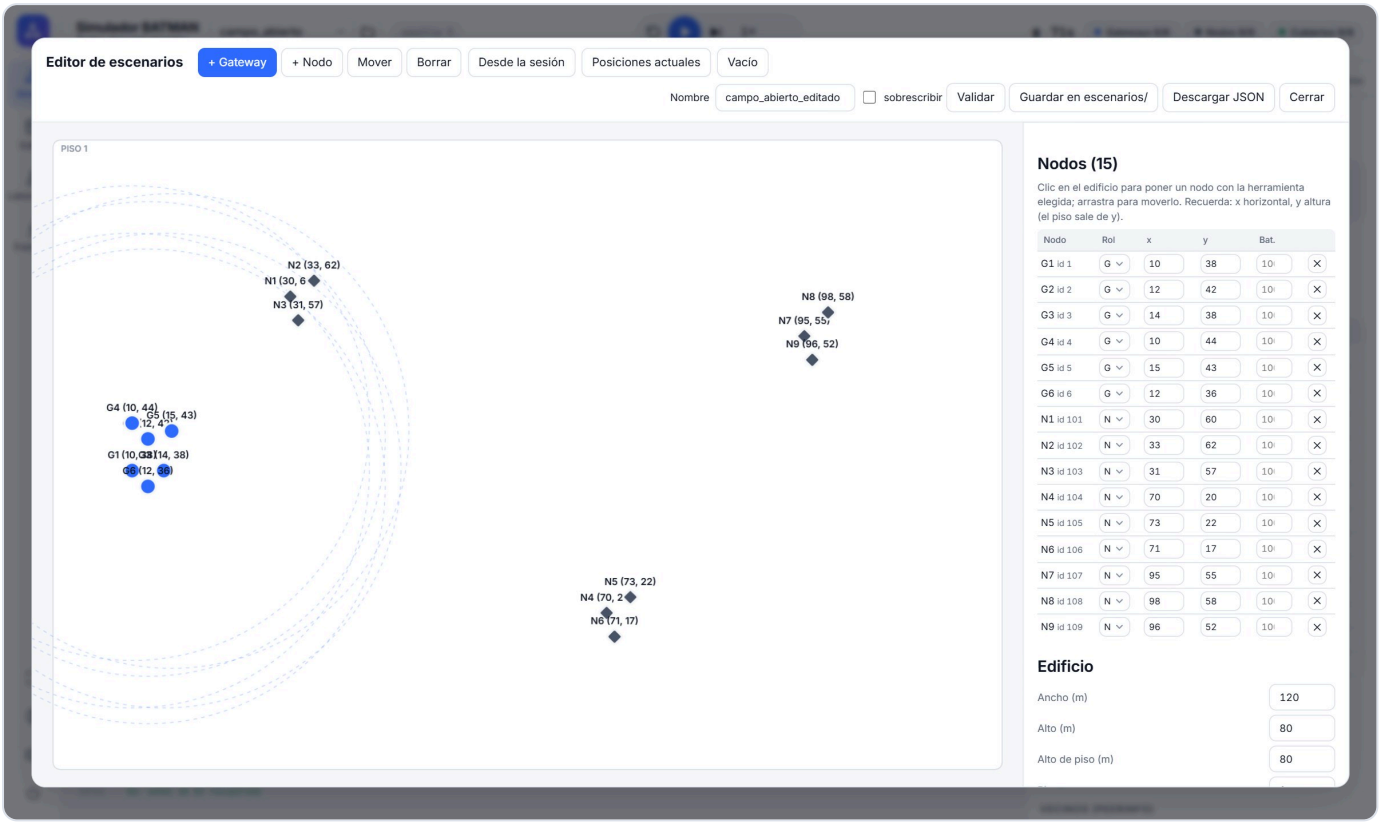
Métrica	Azar	Dirigida	Qué nos dice
Tiempo con la malla partida	42 s	**180 s**	Atacar al crítico parte la red casi todo el tiempo, en las 10 semillas.
Entrega de datos	0.50	**0.28**	Llega casi la mitad de mensajes que con fallos al azar.
Perdidos: sin camino por radio	0.26	**0.60**	La causa principal: físicamente no hay cómo llegar.
Entrega con camino por radio	0.69	0.70	Cuando hay camino, el protocolo rinde igual en los dos casos.
Cobertura media de usuarios	0.93	0.92	**Casi no los distingue**: cada mitad conserva algún Gateway.

La conclusión que sirve para la tesis

Mirando sólo la cobertura, los dos ataques parecen iguales (0.93 contra 0.92). El daño real sólo aparece en el plano de datos y en las particiones. Por eso el simulador mide los dos planos.

11 · EDITOR Y EXTRAS

Editor de escenarios



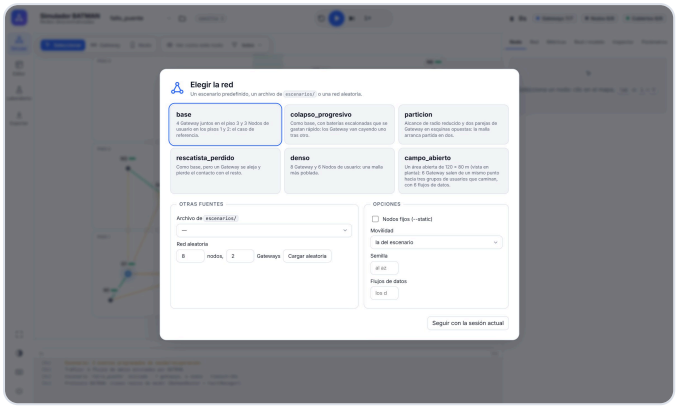
- **+ Gateway / + Nodo / Mover / Borrar:** clic en el plano para poner nodos; arrastrar para moverlos.
- **Desde la sesión** o **Posiciones actuales**: parte de la red que está corriendo. **Vacío**: empezar de cero.
- A la derecha: la tabla de nodos (rol, x, y, batería), el tamaño del edificio, el medio de radio, el protocolo, la movilidad y los **eventos** (caídas, recuperaciones, nodos que se pierden) con su instante.
- **Validar** revisa con las mismas reglas que la terminal; **Guardar en escenarios/** lo deja listo para `--config` y para el laboratorio. Nunca pisa un escenario predefinido.

Exportar (`P`)

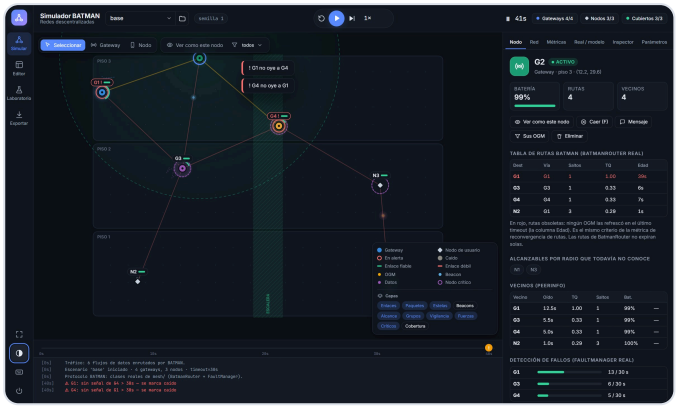
Guarda en `reportes/<escenario>_<fecha>/` : la figura `analisis_red.png` (las mismas gráficas de la sección 9), `reporte.csv/json/txt` con las series y el resumen, y `sesion_web.json` con la semilla, tus intervenciones y el **comando equivalente** de terminal para repetir exactamente la misma corrida.

> Si sólo tumbaste y recuperaste nodos, la sesión se puede repetir por terminal (se exporta como escenario con esos eventos). Si moviste nodos, agregaste o cambiaste parámetros, la interfaz avisa que ya no hay comando equivalente.

Elegir la red, tema oscuro y atajos



Elegir la red (sale al abrir): escenario predefinido, archivo de `escenarios/` o red aleatoria; y opciones de nodos fijos, movilidad, semilla y flujos de datos.



Tema oscuro (escenario *base*). Se cambia con el botón de media luna del riel.



Atajos de teclado (botón de teclado del riel o `?`).

Los atajos más útiles

`Espacio`	Pausa / reanuda
`1` – `9`	Seleccionar el Gateway N
`F` / `G`	Tumbar / recuperar el seleccionado
`L`	Ver como este nodo
`O`	Sólo los OGM del seleccionado
`M`	Mandar un mensaje
`C`	Mapa de cobertura
`R`	Reiniciar (misma semilla)
`P` / `Q`	Exportar / terminar
`Z`	Modo presentación

CHULETA FINAL

Cómo explicarlo en voz alta

En 30 segundos

"Hicimos un simulador a la medida para redes descentralizadas. Simula una red de radios (Gateways) que dan cobertura a usuarios en un edificio, sin una central. Lo especial es que corre el código real del protocolo BATMAN; sólo simulamos el mundo físico. Así podemos tumbar radios, moverlos o cambiar la señal y medir cuánta cobertura se pierde, cuánto tarda la red en darse cuenta y cuántos mensajes siguen llegando."

Demo sugerida (5 minutos)

1. Abrir `python main.py --web` y cargar **fallo_puente** con 6 flujos de datos. Mostrar los pisos, los Gateways y los usuarios.
2. Seleccionar un Gateway y mostrar su **tabla de rutas**; activar **Ver como este nodo**: "cada nodo tiene su propia visión".
3. Mandar un mensaje de N1 a N6 y leer el camino que tomó.
4. Dejar correr hasta los 60 s: cae G6, aparece **RED PARTIDA**; activar **Vigilancia** y esperar las alertas (~90 s).
5. Abrir **Red** (rutas en rojo) y **Métricas** (gráficas 2, 4, 7 y 8).
6. Terminar con el **Laboratorio** y la comparación azar contra dirigida.

Preguntas que suelen hacer

Pregunta	Respuesta corta
¿Por qué no usar ns-3 u OMNeT++?	Esos evalúan un *modelo* del protocolo; aquí corre la implementación real, con un medio que tiene en cuenta los pisos del edificio y con vista por nodo.
¿Es un simulador de rescatistas?	Nació así, pero hoy es de cobertura y resiliencia en redes descentralizadas.
¿Los resultados se pueden repetir?	Sí: misma semilla = misma corrida, en la web y en la terminal (está probado).
¿Qué no modela?	Latencia y colas (cada salto se resuelve dentro del paso de 0.5 s); el ancho de banda es una estimación.
¿Cómo sabe un nodo que otro cayó?	Por silencio: si pasan más de 30 s sin oírlo, lo da por caído. Cada Gateway decide por su cuenta.
¿Por qué el TQ es bajo (≈0.3)?	Porque en los escenarios del Capítulo 5 los enlaces entre pisos pierden bastantes OGM, y el TQ de una ruta multiplica los de cada salto.

Capturas tomadas el 4 de noviembre de 2026 con la interfaz web del repositorio (`main` , commit 5ac2295), escenario *fallo_puente*, semilla 1, 6 flujos de datos; *campo_abierto* semilla 3; *base* semilla 1; lote `lotes/robustez.json` .